

The F1R3Skein Virtual Instrument

Specification, revision 3

F1R3FLY.io

4 September 2026

Abstract

This document specifies the F1R3Skein virtual instrument: the gestural surface through which a musician draws melody out of the digit streams of mathematical constants, and out of which the platform’s two primary artifacts—tunes and performances—are produced.

Revision 3 replaces the instrument’s physical model rather than adjusting it. The two streams now have separate spools, separately mobile, spread apart on the x -axis. Ribbon spools toward the musician and divides into future, present and past. The ribbons carry notches on their inner edges; bringing them together meshes those notches, and the mesh propagates as a wave into the future. A notch pair is a note, the zip front is the playhead, and the wave’s speed is the tempo. Capture is a two-handed act performed against a mount that takes over the holding.

The gesture vocabulary grows from six to nine, each specified here with a diagram. Three consequences are structural and are flagged where they arise: the recogniser needs a trajectory detector as well as pose tests, it needs modes, and the cut carries two positions rather than none. Section 15 lists what remains open.

Contents

1	Scope and conformance	2
1.1	Terminology	2
2	Deployment configurations	3
3	The generative core	3
3.1	Streams	3
3.2	Two cursors, separately mobile	3
3.3	Note mapping	3
4	The playing surface	4
5	Instrument state	4
6	Gesture vocabulary	4
6.1	Pull	5
6.2	Twist	5
6.3	Zip	5
6.4	Halt — stopping the wave	6
6.5	Unzip	6
6.6	Mount	7
6.7	Loop	7
6.8	Snip	8
6.9	Modes and mutual exclusion	9

6.10	Two kinds of detector	9
6.11	Detection parameters	9
6.12	Finger curl must be scale-invariant	9
7	Snip semantics and the tune term	10
7.1	The defect being corrected	10
7.2	Resolution	10
7.3	Consequences	10
7.4	Naming	11
8	Performance recording	11
9	Wire protocol, version 3	11
9.1	Client to engine	11
9.2	Engine to client	12
9.3	Rate discipline	12
10	Visual model	12
11	Audio model	13
12	Non-functional requirements	13
13	Instrumentation	13
14	Conformance checklist	13
15	Decisions sought	14

1 Scope and conformance

In scope: the generative core, the playing surface and its geometry, the instrument’s state model, the gesture vocabulary and its detection criteria, the wire protocol, the visual and audio models, performance recording, and the budgets that make the instrument playable.

Out of scope: identity, galleries, the engagement ledger, breeding and selection, and the on-chain representation of tunes. This document defines what the instrument *produces*.

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are used in the sense of RFC 2119. *M* denotes the musician throughout.

1.1 Terminology

<i>Engine</i>	The process holding authoritative instrument state.
<i>Client</i>	The visionOS application: gesture recognition, rendering, audio.
<i>Stream</i>	One spigot: a constant, an output base, a cursor position.
<i>Spool</i>	The visible source of one stream, far from M on the <i>z</i> -axis.
<i>Ribbon</i>	The visible material spooling from one spool toward M.
<i>Notch</i>	One digit position, presented on a ribbon’s inner edge.
<i>Skein</i>	The pair of ribbons together with their relative registration.
<i>Zip front</i>	The boundary where unmeshed ribbon becomes meshed. The playhead.

Tune A snipped, named artifact.

Performance The recorded trace of a session.

2 Deployment configurations

S1 — Tethered.

Engine on a Mac, client on the headset, TCP over the local network with Bonjour discovery. September test configuration; explicitly a stopgap.

S2 — On-device.

Engine built for `arm64-apple-visionos` as a static library behind a C foreign-function interface. No transport at all. The target.

The protocol of Section 9 is defined over an abstract bidirectional channel; in S1 that is a TCP stream, in S2 a pair of FFI ring buffers. No other part of this specification distinguishes them, and the client **MUST NOT** contain transport-specific logic outside its channel implementation.

The Unix-domain-socket transport **MUST NOT** be used for device testing: a Unix socket is addressed by a path in one host’s filesystem, and the headset is a different host.

3 The generative core

3.1 Streams

The engine **MUST** provide the five constants— π , e , φ , $\ln 2$, Liouville’s—each emitting digits in a configurable base b , $2 \leq b \leq 36$. Emission **MUST** be exact and unbounded. A stream configuration is a pair (c, b) ; with a cursor position p it determines the digit $d(c, b, p)$ uniquely.

The π implementation **SHOULD** move to Bailey–Borwein–Plouffe extraction where the base permits, and **MUST** memoise emitted digits so re-realisation of a visited range is $O(1)$ per digit. Deep positions are exactly what a long session produces.

3.2 Two cursors, separately mobile

The skein holds two configurations and two *independent* cursor positions, p_L and p_R . Independence is the point of the design and **MUST** be preserved: it is what lets M hold a rhythmic motif fixed while advancing pitch, or dually.

Proposition 1 (The instrument performs its own algebra). *Holding one ribbon fixed and meshing a different segment of the other against it takes the durations of one candidate and the pitches of another. That is the crossover operator XPitch of the breeding engine, performed by hand; its dual is $\text{XPitch}(f, m)$ by the rectangular-band identity. Selection and reproduction are not a subsystem bolted onto a music toy—the population does at scale what M does at the bench.*

3.3 Note mapping

One stream governs pitch, the other duration; *twist* exchanges the roles. For the k -th meshed notch pair the engine **MUST** produce $(\Pi(d_{\text{pitch}}), \Delta(d_{\text{duration}}))$. Both maps **MUST** be session parameters, not build constants. Pitch maps **MUST** include major, minor, pentatonic major, pentatonic minor, Dorian, whole tone and chromatic over a configurable root; duration maps **MUST** include musical, linear, exponential and fixed. A stream’s base **SHOULD** match the cardinality of the map it drives.

4 The playing surface

Axes are M's: x is left-right, y is up-down, z is depth away from M.

Each stream has its own spool. The two spools sit at the *same height* and are spread apart on the x -axis, far from M on z . Ribbon spools from each toward M's hands. Each ribbon divides into three sections:

Future still wound on the spool, unseen.

Present between the spool and M's hands, visible.

Past having passed the hands, hanging below them toward the floor.

Each ribbon carries notches on its *inner* edge—the edge facing the other ribbon. Bringing the ribbons together meshes the notches over a section of the present, leaving unmeshed margins near the hands and near the spools. The mount sits between the ribbons, just in front of the hands, and is a pair of clips.

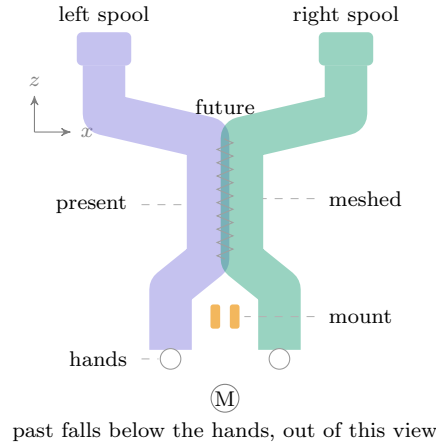


Figure 1: The playing surface, seen from above. Two spools at the same height, spread on x .

A notch pair is a note. Meshing commits notch k on the left to notch k on the right. While unmeshed the ribbons slide independently and M sets the relative phase; once meshed the pairing is fixed. Independent mobility and well-formed pairing are not in tension—they are separated in time by the zip.

5 Instrument state

$$\Sigma = \langle (c_L, b_L, p_L), (c_R, b_R, p_R), \zeta, \mu, \lambda, \Pi, \Delta, \tau, \iota, T \rangle$$

with ζ the zip state (unzipped, or zipped with a front position and a propagation rate), μ the mount state, λ the loop state, τ tempo, ι instrument, T the tune tray. The client MUST NOT hold authoritative state; it renders Σ and emits gestures.

6 Gesture vocabulary

Nine gestures. Each subsection gives the physical form, the effect, and a diagram of the surface during that gesture.

6.1 Pull

M draws one hand toward herself, advancing that stream. Step count is proportional to speed. Pulls are per-hand and independent; this is where the relative phase between the two streams is set.

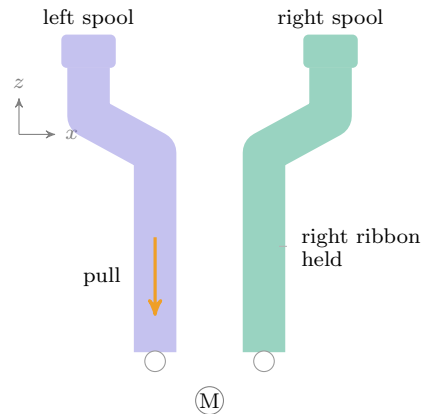


Figure 2: Pull. Ribbons unmeshed; each advances alone. Holding one and pulling the other is how M chooses which rhythm meets which pitches.

6.2 Twist

One hand held clearly above the other, hands apart. Exchanges the two streams, cursors included. Twist MUST be permitted only while unzipped: the mesh commits the pairing, and exchanging roles underneath a committed mesh has no coherent reading.

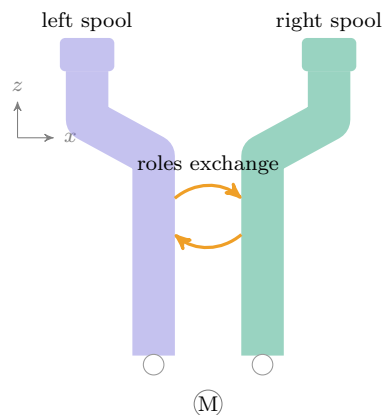


Figure 3: Twist. Which stream governs pitch and which governs duration is swapped. Permitted only while unzipped.

6.3 Zip

M brings the two ribbons together. The notches mesh, and **the mesh keeps going**: the zip front propagates away from M into the future without further input. As the meshed section outgrows the present, the spools appear to recede to supply more ribbon.

The zip front is the playhead. Each notch pair sounds as it engages, so zipping *is* playing, and the propagation rate is the tempo.

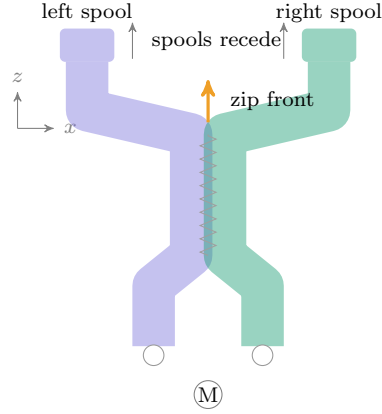


Figure 4: Zip. The wave propagates into the future unaided; the spools retreat to feed it. Each notch pair sounds as it engages.

6.4 Halt — stopping the wave

M looks at the zip front and stops it. The wave freezes, the spools stop receding, and a short unmeshed reserve remains between the meshed section and the spools.

Platform constraint. visionOS does not expose gaze to applications. Hover highlighting is rendered out-of-process by the system and is not reported to the app, and there is no `onHover` equivalent: the application learns nothing until the user commits. A continuous “M looks and the wave halts” therefore cannot be built.

The realisable form preserves the intent. The zip front entity **MUST** carry a `HoverEffectComponent`, so the system highlights it as M looks—costing us nothing and leaking nothing—and M commits with a small pinch, which reaches the app as a spatial tap carrying its 3D location. Her eyes still aim; the pinch is only the trigger, and the zip front is the one thing out there to look at.

Head direction **MUST NOT** be substituted as a gaze proxy. A musician’s head moves constantly and independently of her eyes; the wave would halt every time she glanced at the tray.

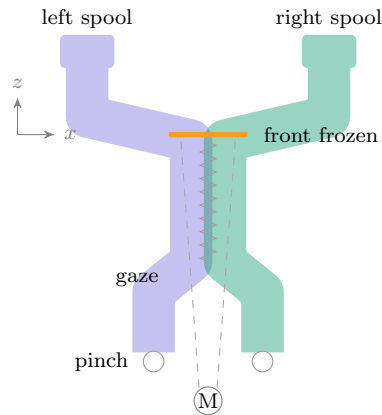


Figure 5: Halt. M looks at the front; the system highlights it; a pinch commits. The unmeshed reserve above the front is what a later zip resumes from.

6.5 Unzip

Separates the ribbons, retiring the mesh from the near end. Unzipping is what lets M re-set the relative phase and mesh a different segment against a held ribbon—the crossover move of

Proposition 1.

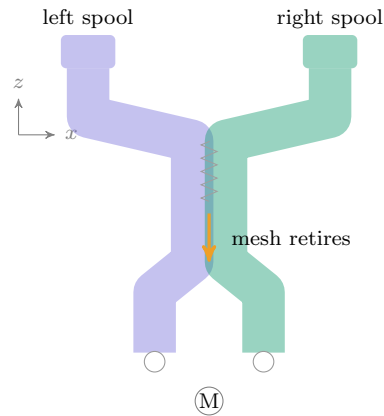


Figure 6: Unzip. The pairing is released from the near end, freeing the ribbons to slide independently again.

6.6 Mount

M slots the unmeshed lead of each ribbon into the clips. **The mount’s job is to take over the holding.** Until this moment each hand has been driving its own ribbon; once clipped, both hands are free to leave their stations and travel out along the meshed band without the assembly collapsing into the past. This is why mounting **MUST** precede snipping.

The mount contributes no boundary to the captured tune.

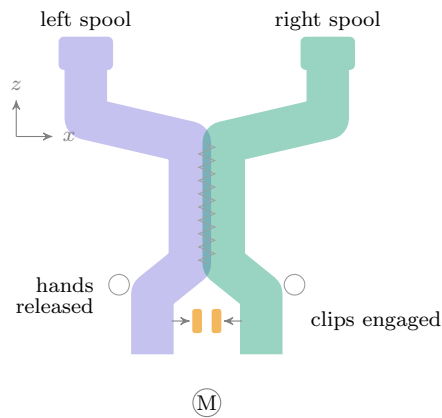


Figure 7: Mount. Two clips take the load; M’s hands are freed for the two-handed snip.

6.7 Loop

A circular finger motion—the “go on” gesture—sets the meshed section looping so M can audition it. The loop span runs from the mount to the halted zip front. Reversing the direction of rotation clears the loop.

Looping needs no new machinery in the representation: repetition is **Seq** of copies, which the monoid already supplies. Loop is a transport control, recorded in the performance trace, and leaves the tune term untouched.

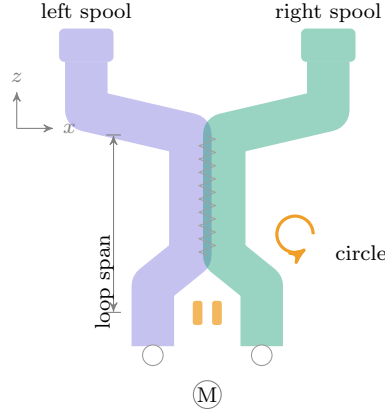


Figure 8: Loop. Mount to halted front. Audition wide, cut narrow.

6.8 Snip

Both hands are involved. One hand is nearest M, the other nearest the spools; what is snipped is the section *between the hands*. Two cut planes free a middle piece that is already gripped at both ends, so M carries it to the tray rather than watching it drop—which is the physical reason a snip needs two hands.

- The near hand supplies i_L and i_R ; the separation of the hands in notches supplies n .
- Roles are assigned by z -ordering at the moment of the gesture, **MUST NOT** by chirality. Either hand may be the near one.
- Both cuts **MUST** land inside the meshed span. Cutting the unmeshed lead would free two loose ribbons rather than a tune.
- The cut **MUST** be previewed: while the pose is held, a marker tracks each hand along the band so M can slide to the right notch before closing.

M is free to cut a smaller section than the full extent from mount to front—that is her choice, and it is why the three decisions are independent: the mount fixes where holding is taken over, halting fixes how much material exists, the cut fixes how much she keeps.

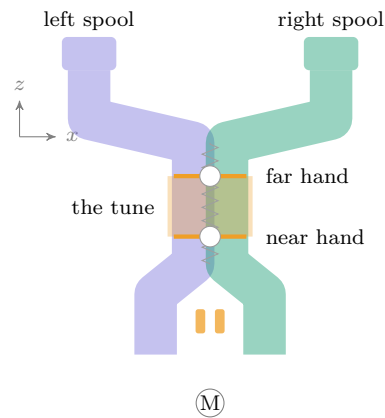


Figure 9: Snip. Two cuts, both inside the mesh. What is between the hands becomes the tune; the piece is held at both ends as it is freed.

6.9 Modes and mutual exclusion

Three rules are new in this revision and one of them is not optional.

Cut mode is mandatory. Moving a hand along z *is* the pull gesture. During a snip M does exactly that with both hands, travelling out along the band to bracket a section. Pull therefore **MUST** be suppressed between mounting and the completion or abandonment of a snip. Without this the streams advance while M is choosing a cut. The current recogniser has no notion of mode—every predicate is evaluated every frame against every hand—so this is more invasive than adding a gesture, and it produces baffling behaviour if discovered late.

Circling must not read as pulling. A circling hand crosses the pull velocity threshold twice per revolution. Loop and pull **MUST** be mutually excluded.

Twist requires unzipped. As §6.2.

The rules retained from revision 2: gestures are edge-triggered with cooldowns and **MUST NOT** re-fire while a pose is held; zip and twist are never simultaneously satisfiable.

6.10 Two kinds of detector

Every gesture in the current recogniser is a per-frame pose test plus a frame counter. The loop gesture is a *trajectory*, and the recogniser **MUST** therefore grow a second kind of detector: a ring buffer of index-fingertip positions, a plane fit over the window, the unwrapped angle accumulated about the centroid, firing when the sweep passes 2π with radius variance and plane residual inside bounds. Direction of rotation comes free and gives loop-set against loop-clear.

6.11 Detection parameters

Every threshold **MUST** live in an external calibration profile, loaded at launch and reloadable without a rebuild. Hardcoded constants **MUST NOT** remain in the recogniser.

Parameter	Governs	Provisional
<code>pull.velocity_min</code>	speed toward M	0.25 m/s
<code>pull.step_divisor</code>	speed to step count	0.15
<code>pull.cooldown</code>	minimum interval	0.08 s
<code>zip.distance</code>	ribbons-together	0.10 m
<code>unzip.distance</code>	ribbons-apart	0.22 m
<code>twist.height_delta</code>	vertical offset	0.06 m
<code>twist.hold_frames</code>	frames before firing	8
<code>scissors.spread</code>	index/middle angle	0.40 rad
<code>scissors.extended</code>	extension floor	<i>to be derived</i>
<code>scissors.curled</code>	curl ceiling	<i>to be derived</i>
<code>loop.min_radius</code>	circle size floor	<i>to be derived</i>
<code>loop.radius_variance</code>	circularity tolerance	<i>to be derived</i>
<code>loop.plane_residual</code>	planarity tolerance	<i>to be derived</i>

6.12 Finger curl must be scale-invariant

The implemented curl metric is $\| \text{tip} - \text{metacarpal} \| / 0.08$ clamped to unity, with “curled” asserted below 0.30. A fully curled adult ring finger measures roughly 0.06–0.08 m tip-to-metacarpal, giving 0.75–1.0. The threshold is unreachable and scissors never fires. The metric is also not scale-invariant: the fixed divisor makes detection depend on hand size.

Curl MUST instead be the interior angle at the proximal interphalangeal joint, or tip-to-wrist distance normalised by that finger’s own summed segment lengths. Thresholds MUST be derived from recorded traces (§13), not chosen by inspection.

7 Snip semantics and the tune term

7.1 The defect being corrected

The engine currently derives a snip from the *left* cursor alone: it computes $from = p_L - w$ and $to = p_L$ for ribbon width w , then advances both fresh spigots to $from$. Since p_L and p_R are independent by design, the stored snippet is $(d(c_L, b_L, k), d(c_R, b_R, k))_k$ over a single index, while what M heard was $(d(c_L, b_L, i), d(c_R, b_R, j))$ with i and j independent.

A snipped tune is, in general, not the tune M heard.

A third representation compounds it: the tray receives the digits actually displayed, while the snippet map receives the re-derived pairs. “The snipped tune” denotes three different things depending on where one looks.

7.2 Resolution

$\text{Snip}(k, i_L, i_R, n)$ denoting $(d(c_L, b_L, i_L + t), d(c_R, b_R, i_R + t))_{t \in [0, n]}$.

The mesh guarantees equal counts on both sides, so a common length n is exact rather than assumed. The near hand supplies i_L and i_R ; the hand separation supplies n . Tray, snippet map and realised audio MUST all derive from this one term.

7.3 Consequences

The Theory of skeins. `Skein.module` currently declares

```
Snip . k:Skein, i:Nat, j:Nat |- "snip" "(" k " ", " i ", " j ")" : Tune;
```

inheriting the single-interval assumption. It becomes

```
Snip . k:Skein, iL:Nat, iR:Nat, n:Nat
  |- "snip" "(" k " ", " iL ", " iR ", " n ")" : Tune;
```

with the empty-snip equation restated as $(\text{Snip } k \ i_L \ i_R \ \text{NZero}) == (\text{Rest})$. A cut placed with the hands together gives $n = 0$ and normalises to `Rest` by that equation.

Twist commutes with snipping, for free.

Proposition 2. *Normalising $\text{Twist}(\text{Wv } l \ r)$ to $\text{Wv } r \ l$ also reinterprets i_L as indexing r , which is what the engine does—twist swaps whole spigots, configuration and cursor together. No additional equation is required.*

A run yields a sequence of tunes. Since the ribbon beyond the far cut stays attached and stays meshed, M can mount again and cut again. The harvested tunes are adjacent intervals over one skein: $\text{Snip}(k, i_L, i_R, n_1)$, then $\text{Snip}(k, i_L + n_1, i_R + n_1, n_2)$, and so on. The relation between a performance and the tunes taken from it is readable off the syntax rather than stored in a link table—the same argument that made tunes terms in the first place.

7.4 Naming

A snip **MUST** suspend nothing: the wave, if running, continues. M is prompted for a name asynchronously, and if none arrives within a timeout the engine **MUST** assign a provisional one rather than discard the capture.

8 Performance recording

Definition 1 (Performance trace). An ordered sequence of timestamped records: an opening declaration of both stream configurations, the maps, tempo and instrument; then every gesture event with its parameters; then a closing timestamp.

The engine **MUST** record a trace for every session without being asked. The streams are deterministic and the state machine is total, so the trace **MUST** replay the session exactly; no audio is stored. Snips **MUST** appear in the trace, so a performance resolves to the tunes harvested from it and each tune resolves back to its performance. Timestamps **MUST** be monotonic at millisecond resolution or better, and records **SHOULD** carry the raw hand-tracking metrics that produced each gesture, for §13.

9 Wire protocol, version 3

Messages **MUST** be newline-delimited JSON, one object per line, each carrying **type** and **"v": 3**. Both sides **MUST** serialise via **Codable** and **serde**; neither side **MAY** construct JSON by string interpolation. The current client escapes only the double quote, so a name containing a backslash produces malformed JSON that the engine silently discards.

9.1 Client to engine

Type	Fields	Note
<code>pull_left</code> , <code>pull_right</code>	<code>steps</code> , <code>velocity</code>	
<code>twist</code>	—	rejected unless unzipped
<code>zip</code>	—	was <code>clap</code>
<code>unzip</code>	—	was <code>unclap</code>
<code>halt</code>	—	stops the wave
<code>mount</code>	—	
<code>loop</code>	<code>on</code>	from rotation direction
<code>scissors</code>	<code>name</code> , <code>near</code> , <code>far</code>	two notch offsets
<code>configure</code>	<code>left</code> , <code>right</code> , <code>pitch_map</code> , <code>duration_map</code> , <code>tempo</code> , <code>instrument</code>	
<code>quit</code>	—	

`scissors` carrying two positions is the substantive change. The gesture currently fires a nameless, locationless event and the engine derives the window itself from the ribbon width—the assumption this revision removes.

9.2 Engine to client

Type	Fields
digits	left, right, left_pos, right_pos
note	pitch, duration, velocity, left_pos, right_pos, notch_index
zip_state	zipped, front, rate, reserve
snip_ack	name, count, i_left, i_right
state	mounted, looping, left_label, right_label, tempo, instrument
status	text
error	code, text

Playback state **MUST** be a field, never inferred from prose. The client currently clears its playing flag on `statusText.contains("stopped")`. Nothing in the client **MAY** parse `status`, which is advisory and human-readable only.

9.3 Rate discipline

The engine currently emits `status` on every frame of its 60 Hz loop, beneath a comment reading “Push status if changed” describing a check the code does not perform. Each message invalidates the entire client panel.

- `status`, `state` and `zip_state` **MUST** be emitted only on change.
- `digits` **MUST** be emitted only when a cursor moves, coalesced to at most 30 Hz.
- `note` **MUST** be emitted per note and **MUST NOT** be coalesced.
- Aggregate steady-state traffic **SHOULD** stay below 200 messages per second.

10 Visual model

Axis correction. `RibbonView.swift` places both ribbons at a shared `RIBBON_X` and separates them by `LEFT_Y`/`RIGHT_Y`—stacked vertically at the same left-right position. Under §4 they separate on x , left spool to the left hand. This **MUST** be corrected before any geometry is tuned.

Patches. Pooled and reused. Text meshes and materials **MUST** be cached per digit value and per depth bucket and **MUST NOT** be regenerated per frame; `PatchEntity.configure` currently calls `MeshResource.generateText` and builds a fresh physically-based material for every patch on every update, which is the dominant cost in the render path.

The three sections **MUST** be visually distinct: future occluded on the spool, present taut and legible, past hanging below the hands and dimmed.

The mesh **MUST** be rendered at the notch scale, and the zip front **MUST** be a distinct, hoverable entity—it is both the playhead and the target of the halt gesture.

Hand ghosts **MUST** mirror the tracked joints. The recogniser receives every hand anchor and discards it; the ghost entity is added to the scene and never updated.

The tray **MUST** display the digits actually snipped, not the live ribbon contents.

11 Audio model

Audio **MUST** be synthesised on the headset. In the current architecture the engine drives CoreMIDI on the Mac, so M pulls a ribbon in front of her face and hears the result from a laptop across the room, after a network hop and a MIDI buffer. That makes the latency figure—the most important measurement of the September sessions—unrepresentative of any shipping configuration.

- The client **MUST** render **note** messages through an on-device sampler.
- Audio **SHOULD** be spatialised so each ribbon sounds from its own position. The two streams are distinct objects in front of M and should be audibly distinct.
- Mac-side MIDI **MAY** be retained for recording to a workstation and **MUST NOT** be the default.

12 Non-functional requirements

Property	Target	Maximum
Notch engagement to audible note	30 ms	50 ms
Gesture to visual response	one frame	two frames
Rendered frame rate	90 fps sustained	—
Digit emission, memoised range	—	1 ms per 1000 digits
Engine to client message rate	$<100\text{ s}^{-1}$	200 s^{-1}
Session length without degradation	45 min	—

Failure modes **MUST** be visible and recoverable. Channel loss **MUST** surface in the panel and retry with backoff, preserving engine state. Denial of hand-tracking authorization **MUST** produce an explanatory message and fall back to panel controls, not a console line. The tray **MUST NOT** silently discard entries; it currently caps at twenty and drops the oldest.

13 Instrumentation

The thresholds are guesses, one is provably unreachable, and there is no data to tune against.

- A debug overlay **MUST** display live metrics per hand: separation, palm velocity, height differential, per-finger extension and curl, accumulated rotation angle, and which predicates are satisfied.
- Raw joint transforms **MUST** be recordable at tracking rate with a manual label marker.
- Thresholds **MUST** be tunable offline and reloadable without a rebuild.

The first half-day of headset time **SHOULD** capture a labelled corpus rather than play. Those traces are the same artifact as a performance trace, so the work is a first delivery of the product rather than a detour from it.

14 Conformance checklist

1. Clean install on device; entitlements and provisioning verified.
2. Hand-tracking authorization requested, granted, denial handled visibly.
3. Engine discovered automatically; connection indicated within five seconds.

4. All nine gestures fire, each confirmed in the debug overlay, before any music is attempted.
5. Hand ghosts visible and tracking.
6. Zip propagates unaided; spools recede; each notch pair sounds from the headset.
7. Halt freezes the front and leaves an unmeshed reserve.
8. Loop set and cleared by rotation direction; loop span runs mount to front.
9. Snip with two hands yields a tune of exactly the bracketed notch count.
10. A snipped tune, re-realised from (k, i_L, i_R, n) , is identical to what was heard.
11. Two successive snips from one run produce adjacent intervals.
12. Pull is suppressed throughout cut mode.
13. 90 fps sustained with sixty patches per ribbon.
14. Tray and performance trace survive an application restart.
15. Gesture traces written to disk.

Items 10 and 11 are the acceptance tests for §7; neither can pass on the current build.

15 Decisions sought

1. **Recede or scroll.** As the wave runs, the spools retreat and the ribbon lengthens. Does recession continue indefinitely, with patches shrinking until unreadable, or stop at a bound after which the world scrolls past—spools held at fixed depth, ribbon flowing toward M? The second keeps digits legible on a long take, but the transition must be invisible or it reads as a glitch.
2. **After the cut.** The band is left in two pieces: a near stub still clipped to the mount, and the remainder still on the spools. Does the stub drop into the past and the remainder get drawn back for a fresh mount, or does something reconnect?
3. **Tempo control.** The wave’s propagation rate is the tempo. What sets it, and can M change it mid-run?
4. **Configuration ordering.** S1 tethered for September with S2 to follow (about three days then about a week), or straight to S2 and accept the risk to the test window?
5. **Modulable maps.** Should the pitch and duration maps be modulable by gesture during a run? If so the tune representation must carry them, which touches the Theory.
6. **Recording consent.** Recording every session by default is proposed here. Confirm, and confirm whether traces leave the device before M chooses to publish.
7. **Naming timeout.** What provisional name, and after how long?